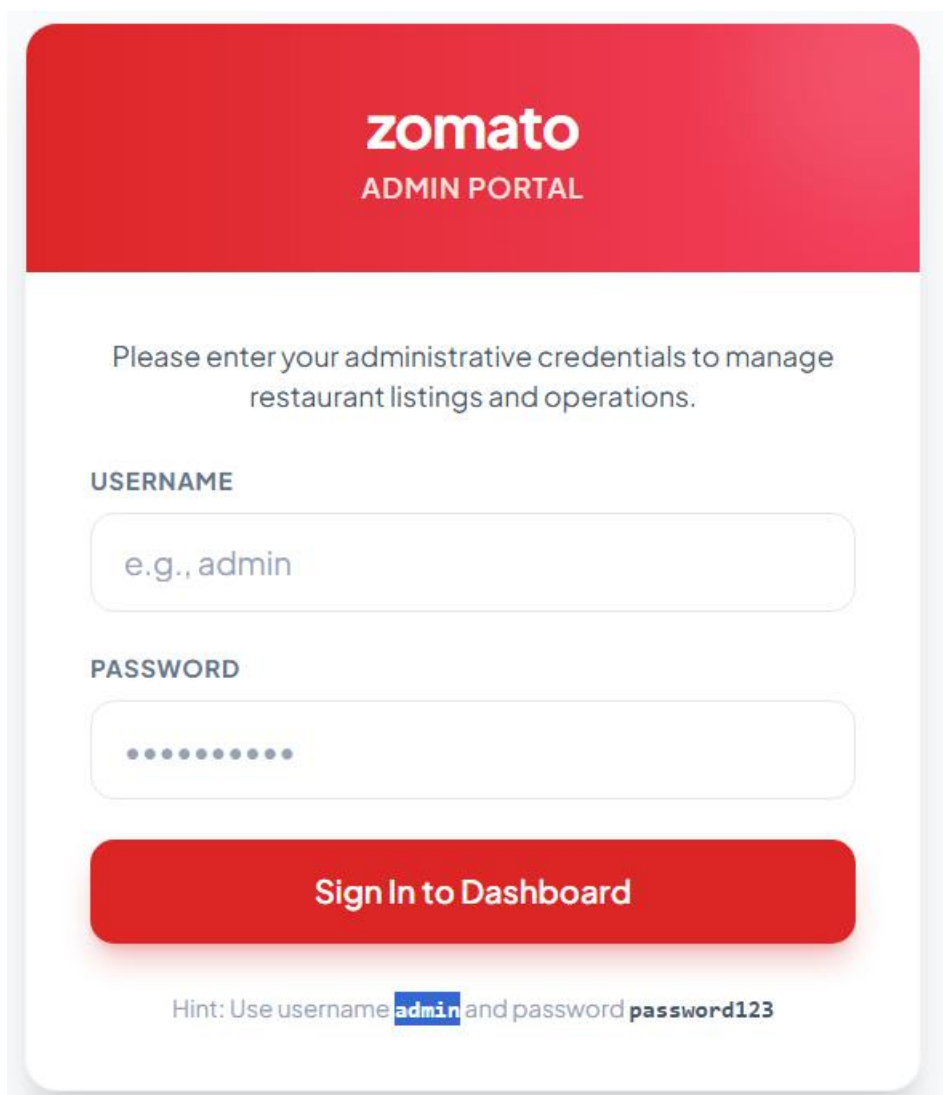


ASSIGNMENT 3

❖ Authentication and Access Control

Task (1):

Login Page:



The image shows a login page for the Zomato Admin Portal. It features a red header with the Zomato logo and the text 'ADMIN PORTAL'. Below the header, there is a white box containing the login form. The form includes a message: 'Please enter your administrative credentials to manage restaurant listings and operations.' followed by two input fields: 'USERNAME' and 'PASSWORD'. The 'USERNAME' field has a placeholder text 'e.g., admin'. The 'PASSWORD' field is masked with dots. Below the input fields is a red button labeled 'Sign In to Dashboard'. At the bottom of the form, there is a hint: 'Hint: Use username **admin** and password **password123**'.

zomato
ADMIN PORTAL

Please enter your administrative credentials to manage restaurant listings and operations.

USERNAME

e.g., admin

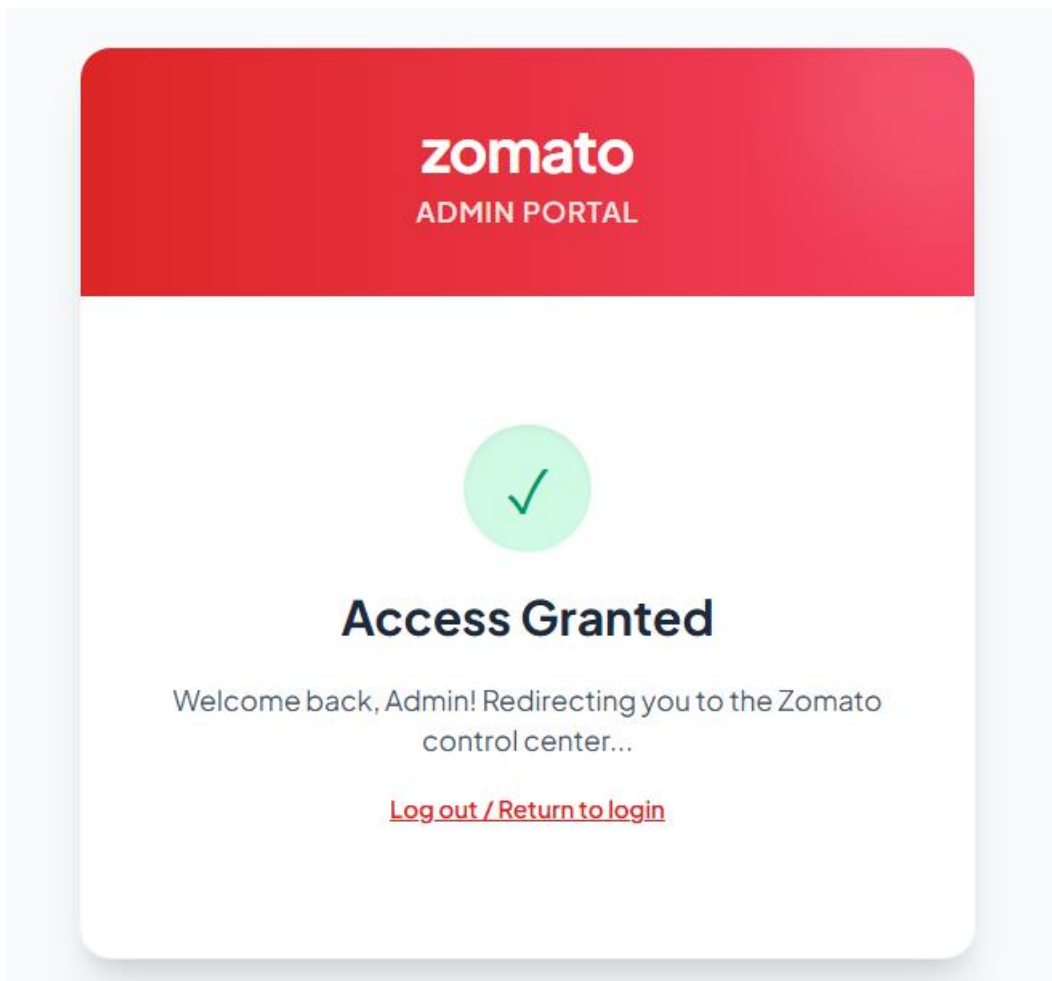
PASSWORD

.....

Sign In to Dashboard

Hint: Use username **admin** and password **password123**

Access Page :



Task (2):

Python Code:

```
def check_access(user_role, resource):  
    permissions = {  
        'customer': ['order_history'],  
        'restaurant_owner': ['menu_edit',  
            'order_history'],  
        'admin_dashboard_user':  
            ['admin_dashboard', 'menu_edit',  
            'order_history'] # or use 'admin'  
    }  
  
    allowed_resources =  
        permissions.get(user_role, [])  
  
    return resource in allowed_resources  
  
print(check_access('customer',  
    'order_history'))  
  
print(check_access('customer', 'menu_edit'))
```

```
print(check_access('restaurant_owner',  
'menu_edit'))
```

```
print(check_access('restaurant_owner',  
'admin_dashboard'))
```

Task (3):

Python Code:

```
import random
```

```
def paytm_login_simulation():
```

```
    print("--- Welcome to Paytm Login ---")
```

```
    stored_password = "password123"
```

```
    password = input("Enter your password  
(hint: password123): ")
```

```
    if password != stored_password:
```

```
        print(" ❌ Incorrect password. Access
```

```
denied.")
```

```
    return
```

```
    print("✓ Password verified successfully!\n")
```

```
    generated_otp =  
    str(random.randint(100000, 999999))
```

```
    print(f"[SMS Simulation] Your Paytm  
verification code is: {generated_otp}")
```

```
    user_otp = input("\nEnter the 6-digit OTP:  
")
```

```
    if user_otp == generated_otp:
```

```
        print("\n👑 Login Successful! Welcome  
back.")
```

```
    else:
```

```
print("\n ✖ Incorrect OTP.  
Authentication failed.")
```

```
if __name__ == "__main__":  
    paytm_login_simulation()
```

Task (4):

Code:

```
from datetime import datetime, timedelta  
  
def get_active_premium_users(users):  
    today = datetime.strptime('2024-06-10',  
                              '%Y-%m-%d')  
    seven_days_ago = today - timedelta(days=7)  
  
    active_premium_users = []
```

```
for user in users:

    if user.get('isPremium'):

        last_login =
datetime.strptime(user.get('lastLogin'), '%Y-
%m-%d')

        if seven_days_ago <= last_login <=
today:

            active_premium_users.append(user)

return active_premium_users

sample_users = [

    {"username": "alice_p", "isPremium": True,
"lastLogin": "2024-06-08"}, # Match
(Premium, 2 days ago)
```

```
    {"username": "bob_free", "isPremium":  
False, "lastLogin": "2024-06-09"}, # No match  
(Not premium)
```

```
    {"username": "charlie_p", "isPremium":  
True, "lastLogin": "2024-05-20"}, # No match  
(Too old)
```

```
    {"username": "diana_p", "isPremium":  
True, "lastLogin": "2024-06-05"} # Match  
(Premium, 5 days ago)
```

```
]
```

```
result =  
get_active_premium_users(sample_users)  
print(result)
```

Task (5):

1. Insecure Direct Object Reference (IDOR / BOLA):

- **The Mistake:** Exposing direct identifiers (like `order_id=4521` or `profile_id=102`) in URLs or API requests and assuming that because a user is logged in, they are allowed to view any order or profile by simply changing the number.
- **Fix / Prevention Tip:** Always validate resource ownership on the server side by verifying that the requested object belongs to the currently authenticated user.

2. Relying on Client-Side Enforcement:

- **The Mistake:** Hiding admin options, delete buttons, or vendor tools in the frontend UI for regular customers, while leaving the underlying backend API endpoints completely unprotected.

3. Missing Function-Level Access Control (Vertical Privilege Escalation):

- **The Mistake:** Failing to restrict administrative or seller-only endpoints (such as `/api/admin/delete-item` or `/api/restaurant/approve`), allowing a standard customer account to execute them directly.

4. Mass Assignment (Parameter Over-Posting):

- **The Mistake:** Binding incoming HTTP request data directly to database models during profile updates, allowing malicious users to inject fields like `is_admin: true` or `is_premium: true` into their payload.

5. Unrestricted File and Media Access:

- **The Mistake:** Storing private invoices, user receipts, or photos in publicly guessable cloud storage paths or serving them via raw filenames.